CLUBS EVENTS STUDY GROUPS MARKETPLACE LOST & FOUND POLLS NAVIGATION

SOFTWARE REQUIREMENTS SPECIFICATION

CampusConnect

Unified Student Engagement Platform

Document Version 1.0 · Draft for Faculty Review

Document Type	Software Requirements Specification (IEEE 830 based)
Project Title	CampusConnect – Unified Student Engagement Platform
Prepared For	Frontend Engineering
Prepared By	CampusConnect Project Team
Date	16 August 2026
Version	1.0

Table of Contents

1. Introduction

1.1 Purpose

This document specifies the software requirements for **CampusConnect – Unified Student Engagement Platform**, a responsive frontend web application built to consolidate everyday campus-life activities — clubs, events, study groups, announcements, a peer marketplace, lost & found reporting, polls, and campus navigation — into a single, coherent digital experience. The purpose of this Software Requirements Specification (SRS) is to define, in unambiguous and testable terms, what the system shall do, who will use it, and the constraints under which it must operate, so that it can guide UI/UX design, implementation, testing, and final evaluation by faculty.

1.2 Document Conventions

This document follows a structure adapted from the IEEE 830-1998 recommended practice for software requirements specifications. Requirements are uniquely identified using the pattern **FR-[Module]-[Number]** for functional requirements (e.g., **FR-EVT-02**) and **NFR-[Category]-[Number]** for non-functional requirements (e.g., **NFR-PERF-01**). The keywords **shall** denote mandatory requirements, **should** denote recommended but non-mandatory requirements, and **may** denote optional requirements, consistent with standard requirements-engineering usage.

1.3 Intended Audience and Reading Suggestions

- **Faculty / Evaluators** – read Sections 1, 2, 3 and 7 for scope, features, and evaluation criteria.
- **Student Developers** – read Sections 3, 4, 5, 6 and 8 for functional detail, interface, and architecture.
- **UI/UX Designers** – focus on Sections 3 and 4 for screen-level and interaction requirements.
- **Testers / QA** – focus on Sections 3, 5 and 9 for testable requirements and acceptance criteria.
- **Future Backend/API Developers** – focus on Sections 6 and 7 for data and integration planning.

1.4 Project Scope

CampusConnect is scoped as a **frontend-first, mock/local-data-driven web application** for a single academic institution. In its current phase, the system will be built using modern frontend technologies (HTML5, CSS3, JavaScript/React) with data simulated through local JSON files or browser storage, so that the full user experience can be demonstrated without requiring a production backend. The architecture is deliberately designed so that a REST/GraphQL API and a relational or document database can be plugged in during a later phase without redesigning the UI layer. The system is **not** intended, in this phase, to process real payments, send real push notifications, or integrate with the institution's official ERP/LMS — these are called out explicitly as future-phase or out-of-scope items in Section 2.6.

In scope for this phase: student authentication (mock), club discovery and membership, event discovery and RSVP, study group formation, campus announcements, a peer-to-peer marketplace for listings, a lost & found board, simple polls/surveys, and an interactive campus navigation view, all wrapped in one unified, responsive interface with a consistent design system.

1.5 References

- IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications.
- IEEE Std 29148-2011 – Systems and Software Engineering – Life Cycle Processes – Requirements Engineering.
- W3C Web Content Accessibility Guidelines (WCAG) 2.1, Level AA.
- Material Design and modern responsive web design guidelines (for UI/UX reference).

2. Overall Description

2.1 Product Perspective

CampusConnect is a new, standalone, self-contained product. It is not a modification of an existing system, though conceptually it plays the role that several disconnected tools (WhatsApp groups, notice boards, spreadsheets, and paper flyers) currently play on most campuses. The product is delivered as a single-page or multi-view responsive web application, accessible through any modern browser on desktop, tablet, or mobile, without requiring installation.

2.2 Product Features (Summary)

At a high level, CampusConnect provides the following feature groups, each detailed as a System Feature in Section 3:

Feature Group	Description
Student Authentication & Profile	Sign up, log in, and maintain a personal profile with interests and activity history.
Clubs Directory	Browse, search, and join/follow student clubs and societies.
Events Hub	Discover campus events, view details, and RSVP/register.
Study Groups	Create or join subject-wise study groups with schedules.
Announcements	View categorized, time-stamped campus and department announcements.
Campus Marketplace	Post and browse peer-to-peer listings (books, gadgets, notes, etc.).
Lost & Found	Report and search lost or found items with photos and descriptions.
Polls & Surveys	Participate in and view results of quick campus polls.
Campus Navigation	View an interactive campus map with key locations highlighted.

2.3 User Classes and Characteristics

User Class	Definition	Characteristics & Access
Student (Primary User)	Any enrolled student.	Browses/joins clubs and events, posts listings, reports lost & found items, votes in polls, views map. Expected to be a casual, non-technical user on a personal device.
Club/Event Organizer	Student with elevated privileges for their club.	Creates and manages events and announcements for their club; moderate familiarity with digital tools expected.

Administrator / Moderator	Faculty coordinator or student council admin.	Approves club/event listings, moderates marketplace and lost & found posts, publishes institution-wide announcements and polls.
Guest / Visitor	Unauthenticated user.	Can view public content (events, announcements, club directory) in read-only mode to encourage sign-up.

2.4 Operating Environment

- **Client:** Modern evergreen web browsers — Chrome, Firefox, Edge, Safari (latest two major versions).
- **Devices:** Desktop/laptop, tablet, and smartphone, via a single responsive codebase (mobile-first breakpoints).
- **Hosting (development phase):** Static hosting suitable for frontend builds (e.g., Netlify, Vercel, GitHub Pages) or a local development server.
- **Data layer (current phase):** Local JSON data files / browser localStorage acting as a mock data source; structured to mirror a future REST API response shape.

2.5 Design and Implementation Constraints

- The system must be implementable by a small student team (3–5 members) within a single academic semester.
- The technology stack shall be limited to widely taught, well-documented frontend technologies (HTML5, CSS3, JavaScript, and a component framework such as React), avoiding niche or paid tooling.
- The UI shall follow one consistent design system (colour palette, typography, spacing, component library) across all modules.
- No real financial transactions, real-time push notifications, or third-party campus-system integrations are permitted in this phase (see Section 2.6).
- The application shall be responsive and functional without a persistent backend connection, using mock/local data.

2.6 Assumptions and Dependencies

- It is assumed that a future phase will introduce a real backend (REST/GraphQL API) and database; the frontend data layer is structured to make this transition low-risk.
- It is assumed that user authentication in this phase is simulated (mock login) and does not need to meet production-grade security certification, though secure-by-design practices are still followed.
- The project depends on the availability of open-source component libraries and icon sets that are compatible with the chosen framework.
- It is assumed the institution provides (or the team recreates, for demonstration) a basic campus map image or coordinate layout for the Navigation module.

3. System Features (Functional Requirements)

This section defines each system feature (module), its description, priority, and the specific functional requirements (FRs) that must be satisfied. Priority is expressed as **High** (core, must-have for evaluation), **Medium** (important, expected), or **Low** (nice-to-have, stretch goal).

3.1 Student Authentication & Profile

Priority: High	**Module Code:** AUTH

Allows a student to create an account, log in, and manage a personal profile that personalises the CampusConnect experience (interests, joined clubs, activity history).

ID	Functional Requirement
FR-AUTH-01	The system shall allow a new user to register using name, college email, and password.
FR-AUTH-02	The system shall validate that the email follows the institution's domain pattern before allowing registration.
FR-AUTH-03	The system shall allow a registered user to log in and log out.
FR-AUTH-04	The system shall allow a user to view and edit profile details (name, year, branch, interests, avatar).
FR-AUTH-05	The system shall display a personalised dashboard showing the user's joined clubs, RSVP'd events, and recent activity.
FR-AUTH-06	The system shall allow guest/unauthenticated users to browse public content in read-only mode.

3.2 Clubs Directory

Priority: High	**Module Code:** CLB

Enables students to discover student clubs and societies, view their details, and join or follow them to receive related updates.

ID	Functional Requirement
FR-CLB-01	The system shall display a searchable, filterable directory of all campus clubs by category (technical, cultural, sports, social, etc.).
FR-CLB-02	The system shall show a club profile page with description, category, member count, and upcoming events.
FR-CLB-03	The system shall allow a logged-in student to join or leave a club.
FR-CLB-04	The system shall allow a Club Organizer to edit their club's profile and post club-specific announcements.
FR-CLB-05	The system shall list the clubs a student has joined on their profile/dashboard.

3.3 Events Hub

Priority: High **Module Code:** EVT

Centralises discovery of campus events — workshops, fests, seminars, competitions — with details and RSVP tracking.

ID	Functional Requirement
FR-EVT-01	The system shall display an events list/calendar view with filters for date, category, and organizing club.
FR-EVT-02	The system shall show an event detail page with date, time, venue, description, and organizer.
FR-EVT-03	The system shall allow a logged-in student to RSVP or register interest for an event.
FR-EVT-04	The system shall allow a Club/Event Organizer to create, edit, and cancel events for their club.
FR-EVT-05	The system shall highlight upcoming events (within the next 7 days) on the student dashboard.

3.4 Study Groups

Priority: Medium **Module Code:** SG

Helps students form and join subject-wise or exam-wise study groups with shared schedules and basic coordination details.

ID	Functional Requirement
FR-SG-01	The system shall allow a student to create a study group by specifying subject, description, and meeting schedule.
FR-SG-02	The system shall display a searchable list of active study groups filterable by subject/course.
FR-SG-03	The system shall allow a student to join or leave a study group.
FR-SG-04	The system shall display group member count and next scheduled session on the group card.

3.5 Announcements

Priority: High **Module Code:** ANN

Provides a single, categorised feed of official and club-level announcements, replacing scattered notice boards and message groups.

ID	Functional Requirement
FR-ANN-01	The system shall display announcements in reverse-chronological order with category tags (Academic, Administrative, Club, Event).
FR-ANN-02	The system shall allow filtering of announcements by category and date range.
FR-ANN-03	The system shall allow an Administrator to publish institution-wide announcements.

FR-ANN-04	The system shall allow a Club Organizer to publish announcements scoped to their own club's followers.
FR-ANN-05	The system shall visually distinguish unread announcements from read ones for a logged-in user.

3.6 Campus Marketplace

Priority: Medium **Module Code:** MKT

A peer-to-peer listing board for students to buy, sell, or exchange items such as books, gadgets, and study notes within the campus community.

ID	Functional Requirement
FR-MKT-01	The system shall allow a logged-in student to create a listing with title, category, price, description, condition, and photo(s).
FR-MKT-02	The system shall display listings in a searchable, filterable grid (by category, price range, and keyword).
FR-MKT-03	The system shall show a listing detail page with seller contact option (e.g., in-app message placeholder or email link).
FR-MKT-04	The system shall allow the listing owner to mark a listing as ‘Sold’ or delete it.
FR-MKT-05	The system shall allow an Administrator to remove listings that violate marketplace guidelines.

3.7 Lost & Found

Priority: Medium **Module Code:** LNF

A dedicated board for reporting and searching lost or found items on campus, improving the odds of items being reunited with their owners.

ID	Functional Requirement
FR-LNF-01	The system shall allow a user to report a lost item or a found item with description, category, location, date, and photo.
FR-LNF-02	The system shall display lost and found reports in two clearly separated, filterable lists.
FR-LNF-03	The system shall allow a user to mark their own report as ‘Resolved’ once the item is returned.
FR-LNF-04	The system shall allow searching lost & found reports by keyword and location.

3.8 Polls & Surveys

Priority: Low **Module Code:** POLL

Enables quick, lightweight polling for student opinions, event feedback, or informal campus decisions.

ID	Functional Requirement
FR-POLL-01	The system shall allow an Administrator or Club Organizer to create a poll with a question and 2–6 options.

FR-POLL-02	The system shall allow each logged-in student to cast one vote per poll.
FR-POLL-03	The system shall display live (or session-refreshed) aggregated results as a simple bar/percentage chart.
FR-POLL-04	The system shall prevent a user from voting more than once on the same poll.

3.9 Campus Navigation

Priority: Low **Module Code:** NAV

Provides an interactive, simplified campus map to help students (especially newcomers) locate key buildings and facilities.

ID	Functional Requirement
FR-NAV-01	The system shall display an interactive campus map with labelled markers for key locations (blocks, library, canteen, hostels, admin office).
FR-NAV-02	The system shall allow a user to search for a location by name and highlight it on the map.
FR-NAV-03	The system shall show a short info card (name, category, short description) when a marker is selected.

4. External Interface Requirements

4.1 User Interfaces

- A consistent, responsive layout with a persistent top navigation bar (Home, Clubs, Events, Study Groups, Marketplace, Lost & Found, Polls, Map, Profile).
- A unified green-accented design system: consistent colour palette, typography scale, spacing, card components, and iconography across all modules.
- Mobile-first responsive breakpoints for phone, tablet, and desktop, with a collapsible navigation menu on small screens.
- Accessible components: sufficient colour contrast, visible focus states, semantic HTML, and alt-text on images, targeting WCAG 2.1 AA where feasible.
- Standard feedback patterns: loading states, empty states (e.g., 'No events this week'), and confirmation dialogs for destructive actions (delete listing, cancel event).

4.2 Hardware Interfaces

The application has no direct hardware interface requirements. It shall run on any standard consumer device (desktop, laptop, tablet, or smartphone) equipped with a modern web browser and an internet or campus network connection. Device camera access may optionally be used (via standard browser file-upload/camera APIs) for attaching photos to marketplace and lost & found listings.

4.3 Software Interfaces

Interface	Description
Mock/Local Data Layer	Local JSON files or browser localStorage simulating API responses for clubs, events, listings, polls, etc.
Future REST/GraphQL API	Planned integration point (Phase 2) for a real backend service; frontend data-access layer is isolated to ease this swap.
Map Rendering Library	A lightweight JS mapping/diagram library (or a custom SVG campus map) for the Campus Navigation module.
Icon / Component Library	An open-source icon set and UI component library consistent with the chosen frontend framework.

4.4 Communication Interfaces

In the current phase, all communication is client-side (no live network protocol beyond standard HTTPS delivery of static assets). When the Phase 2 backend is introduced, the system shall communicate over HTTPS using JSON payloads via RESTful endpoints, secured with token-based authentication (e.g., JWT).

5. Non-Functional Requirements

5.1 Performance Requirements

ID	Requirement
NFR-PERF-01	Initial page load shall complete within 3 seconds on a standard broadband/campus Wi-Fi connection.
NFR-PERF-02	Navigating between modules (Clubs, Events, Marketplace, etc.) shall not exceed 1 second of perceived delay, given client-side routing.
NFR-PERF-03	List views (events, listings, announcements) shall support at least 200 mock records without noticeable UI lag.

5.2 Safety Requirements

As a campus information and coordination platform (not a safety-critical or medical system), formal safety requirements are limited. The system shall, however, clearly label the Lost & Found and Marketplace modules with guidance to meet in safe, public campus locations for item exchange, and shall avoid displaying sensitive personal contact information without user consent.

5.3 Security Requirements

ID	Requirement
NFR-SEC-01	Passwords (even in the mock authentication layer) shall not be stored or displayed in plain text in the UI or client-side logs.
NFR-SEC-02	Only authenticated users shall be able to create, edit, or delete content (listings, events, posts) they own.
NFR-SEC-03	Role-based access shall restrict Administrator-only actions (e.g., publishing institution-wide announcements) from regular students.
NFR-SEC-04	All user-submitted text fields shall be sanitised/escaped before rendering, to prevent injection of malicious scripts (XSS).

5.4 Software Quality Attributes

Attribute	Requirement / Target
Usability	New users shall be able to find and use a core feature (e.g., RSVP to an event) within 3 clicks/taps from the homepage.
Responsiveness	The UI shall render correctly on screen widths from 320px (mobile) to 1920px (desktop) without horizontal scrolling or broken layout.
Maintainability	The codebase shall use a modular, component-based structure so individual modules (Clubs, Events, etc.) can be updated independently.
Portability	The application shall run identically across Chrome, Firefox, Edge, and Safari without browser-specific code branches.
Attribute	**Requirement / Target**
Scalability (design-level)	The mock data layer shall be structured so it can be replaced by real API calls with minimal changes to UI components.

5.5 Usability & Accessibility

- Consistent iconography and colour-coding for each module to build quick visual recognition.
- Keyboard navigability for primary actions (forms, buttons, links) and visible focus outlines.
- Descriptive alt-text for all meaningful images (event banners, listing photos, map icons).
- Clear empty and error states with actionable guidance (e.g., ‘No results — try a different filter’).

6. System Architecture (High-Level)

6.1 Architectural Style

CampusConnect follows a **component-based, layered frontend architecture**, chosen for its suitability to a student team and its clean separation of concerns:

Layer	Responsibility
Presentation Layer	Reusable UI components (navigation, cards, forms, modals) built with a component framework (e.g., React) and a shared design system.
State / Application Layer	Manages application state — current user, active filters, form state — using framework-native state management (e.g., Context API / hooks) or a lightweight state library.
Data Access Layer	A thin abstraction (service/repository functions) that currently reads local JSON/localStorage and is designed to be swapped for real API calls in Phase 2 without touching UI components.
Mock/Future Data Layer	Local JSON files structured to mirror a future REST API's response shape (resources: users, clubs, events, studyGroups, announcements, listings, lostFoundReports, polls, locations).

6.2 High-Level Component Diagram (Textual)

The diagram below is represented in text form for documentation purposes; it should be redrawn as a formal architecture diagram in the accompanying project presentation.

```
[ Browser / Client Device ] | v [ Presentation
Layer: Navbar, Dashboard, Module Views
(Clubs, Events, Study Groups, Announcements, Marketplace, Lost & Found,
Polls, Map) ] | v [ Application/State Layer: Auth state, filters, form state,
notifications ] | v [ Data Access Layer (service functions) ] | v [ Phase 1:
Local JSON / localStorage ] <-- swappable --> [ Phase 2: REST API + Database
]
```

6.3 Module-to-Screen Mapping

Screen	Key Contents / Actions
Home / Dashboard	Personalised feed: upcoming events, unread announcements, joined clubs/groups summary.

Clubs	Directory grid + Club detail page + Join/Leave action.
Events	Calendar/list view + Event detail page + RSVP action + Organizer create/edit form.
Study Groups	Group list + Create group form + Group detail with schedule.
Announcements	Filterable feed with category chips.
Marketplace	Listing grid + Listing detail + Create listing form.
Lost & Found	Two-tab (Lost / Found) list + Report form + Detail view.
Polls	Active poll list + Vote screen + Results view.
Campus Map	Interactive map view + Search + Location info card.
Profile	User details, joined clubs, RSVP'd events, own listings/reports.

7. Data Requirements (Future Database & API Planning)

Although the current phase uses mock/local data, the entities below are defined now so that a future relational (e.g., PostgreSQL/MySQL) or document (e.g., MongoDB) database, and a corresponding REST API, can be implemented directly from this specification with minimal rework.

7.1 Core Data Entities

Entity	Key Attributes
User	id, name, email, passwordHash, role (student/organizer/admin), branch, year, interests[], avatarUrl
Club	id, name, category, description, logoUrl, memberIds[], organizerId
Event	id, clubId, title, description, dateTime, venue, category, rsvpUserIds[]
StudyGroup	id, subject, description, schedule, creatorId, memberIds[]
Announcement	id, title, body, category, scope (global/club), authorId, timestamp
MarketplaceListing	id, sellerId, title, description, category, price, condition, photos[], status
LostFoundReport	id, reporterId, type (lost/found), title, description, location, date, photoUrl, status
Poll	id, question, options[], createdBy, votes[] (userId, optionId), expiresAt
CampusLocation	id, name, category, coordinates (x, y or lat/lng), description

7.2 Notes for Phase-2 API Design

- Each entity above maps to one REST resource collection (e.g., **/api/events**, **/api/marketplace**).
- Relationships (User–Club membership, User–Event RSVP, User–Poll vote) shall be modelled as join tables in a relational database or embedded arrays in a document database.
- Authentication should move from mock login to token-based auth (JWT) with password hashing (e.g., bcrypt) in Phase 2.
- Pagination and filtering parameters (category, date range, keyword) used by the mock data layer in Phase 1 should be mirrored as query parameters in the Phase 2 API for a smooth transition.

8. Use Case Summary

The table below summarises the primary use cases across actors. Detailed use-case flows (preconditions, main flow, alternate flow, postconditions) should be developed for each High-priority item during the design phase and included as an appendix in the final project report.

ID	Actor	Use Case
UC-01	Student	Register and log in to CampusConnect.
UC-02	Student	Browse and join a club.
UC-03	Student	Discover an event and RSVP.
UC-04	Student	Create or join a study group.
UC-05	Student	Post a marketplace listing.
UC-06	Student	Report a lost or found item.
UC-07	Student	Vote in an active poll.
UC-08	Student	Search for a location on the campus map.
UC-09	Club Organizer	Create an event and publish a club announcement.
UC-10	Administrator	Publish an institution-wide announcement and moderate content.

9. Testing Considerations

9.1 Testing Approach

- **Unit testing** of individual UI components (forms, cards, filters) using a framework-appropriate test runner (e.g., Jest + React Testing Library).
- **Integration testing** of module flows end-to-end within the mock data layer (e.g., create listing → appears in marketplace grid → can be marked sold).
- **Usability testing** with a small group of student volunteers to validate navigation clarity and task completion time.
- **Cross-browser and responsive testing** across the target browser matrix and screen-size breakpoints defined in Section 5.4.

9.2 Sample Acceptance Criteria (Illustrative)

Requirement	Acceptance Criterion
FR-EVT-03	Given a logged-in student on an event detail page, when they click ‘RSVP’, then the button state changes to ‘Going’ and the event appears under ‘My Events’ on their dashboard.
FR-MKT-01	Given a logged-in student on the ‘Create Listing’ form, when they submit valid required fields, then the new listing appears at the top of the Marketplace grid within the same session.
FR-POLL-04	Given a student who has already voted on a poll, when they revisit that poll, then the voting options are disabled and their previous choice is shown.

10. Appendix

10.1 Glossary

Term	Definition
RSVP	“Répondez s’il vous plaît” – a user's confirmed intent to attend an event.
Mock Data	Simulated, locally-stored data used in place of a live backend/database during development.
Frontend-first	A development approach that builds and validates the UI/UX layer before backend integration.
Module	A self-contained functional area of the application (e.g., Events, Marketplace).
Organizer	A student user granted elevated privileges to manage a specific club's content.

10.2 Assumptions Log

- The project is evaluated primarily on frontend completeness, UI/UX quality, and requirements traceability, not on production backend security.
- A future team or later semester phase may implement the Phase-2 API and database described in Section 7.
- Institution-specific branding (logo, colours) can be swapped into the existing green design system without structural changes.

10.3 Document Revision History

Version	Date	Description
1.0	16 August 2026	Initial complete draft prepared for faculty review.

End of Document — CampusConnect Software Requirements Specification, Version 1.0.